height

Outline

Contents

1 Introduction

Motivation & Core Research Question

- tk

2 Data Cleaning

Motivation & Core Research Question

- tk

3 Explorative Data Analysis (EDA)

Motivation & Core Research Question

- tk

4 Model Fitting

The next step of the process when working with a neural network is model fitting. Model Fitting is the process by which a machine learning model adjusts its internal parameters to minimize the discrepancy between its predictions and the observed data. In supervised learning, we define a loss function $L(\theta)$ that measures this error—where θ denotes all trainable parameters and then optimize such that:

$$\theta \leftarrow \theta - \eta \, \nabla_\theta L(\theta),$$

using gradient-based methods such as stochastic gradient descent.

Within neural networks, backpropagation efficiently computes $\nabla_\theta L$ by applying the chain

```
{'activation': 'relu',
 'alpha': 0.05,
 'hidden_layer_sizes': (120, 80, 40),
 'learning_rate': 'constant',
 'learning_rate_init': 1,
 'max_iter': 100,
 'solver': 'adam'}
Accuracy: 0.84
```

This then, is the best "fitting" of the possible combinations of the various hyperparameter for the neural network. This set of hyperparameters is also quite excellent because, as will be discussed later, we have an accuracy score that is relatively good; not low, but also not so high that we need to be concerned about the risk of overfitting on the data. This is most apparent in the statistics for our F1 tests for the Logistic Regession and our Neural Network as will be discussed in the following diagonostics section of the paper.

5 Model Diagnostics

The next step in the process of researching with a neural network once the network is trained, is Model Diagnostics. Model Diagnostics is the process by which we, having done our Data Cleaning, Explorative Data Analysis, and Model Fitting, trust but verify that our model has learned meaningful patterns rather than memorizing the random noise present in any real work data we might use. There are generally several ways to go about this process depending on the particular model, but in this section we cover loss-curve analysis, hyperparameter validation, error-propagation checks and F1-score.

The first part of our Model Diagonistics is to plot the training loss over iterations to ensure smooth decay and detect plateaus or oscillations:

$$L^{(t)} = \frac{1}{2N} \sum_{i=1}^{N} \left(y_i - \hat{y}_i^{(t)}\right)^2,$$

where $\hat{y}_i^{(t)}$ is the network's output at iteration t. A monotonically decreasing loss curve indicates stable learning; sudden spikes or flat regions suggest learning-rate issues or vanishing gradients. We performed this test using the following code:

```
plt.plot(clf.loss_curve_)
plt.title{Loss Curve}
plt.xlabel{Iterations}
plt.ylabel{Cost}
plt.show()
```

Generally, what we can observe from our loss-curve gradient is a slightly wiggly but otherwise monotonically decreasing curve as we would hope to see, showing that our model has stable learning over many generations.

5.1 Hyperparameter Tuning

The next step in our model diagnostics is that of a hyperparameter tuning. In short, hyperparameter tuning is looking for those provided hyperparameters that give use the best accuracy. To do this, we make use of the GridSearchCV, which will go through the process automatically. GridSearchCV systematically explores combinations of layer sizes, activation functions, solvers and regularization strengths. We examine cross-validation accuracy and standard deviation to choose a model that generalizes well:

- `hidden_layer_sizes`: (150,100,50), (120,80,40), (100,50,30)
- `activation`: logistic vs. relu
- `solver`: sgd vs. adam
- `alpha`: 1e–4, 5e–2, 1
- learning-rate schedules and initial rates

After fitting, used the following code to produce the accuracy scores from the generated best parameters:

```
print(grid.best_params_)
print("Accuracy: {:.2f}".format(
    accuracy_score(y_test, grid.predict(X_test))
))
```

Which in turn gave us the final output for our hyperparameter tuning.

5.2 Custom Backpropagation Checks

To validate that gradients and weight updates behave correctly within our model, we re-implement a minimal two-layer MLP. At each sample we compute:

$$\delta^2 = (y - a^2)\,\sigma'(z^2), \qquad \delta^1 = \left(W^2\,\delta^2\right) \circ \sigma'(z^1),$$

and update $W \leftarrow W + \eta\, a\, \delta$, $b \leftarrow b + \eta\, \delta$. With convergence signaled when the epoch loss falls below a small tolerance. This is a rather important step in the overal process because in a real world setting, it allows us to insure that our analytic gradients match our numerical approximations as well as make sure that our loss function is backpropagating meaningful infomation.

5.3 F1-score

The usefulness of the F1-score is an important test within our model diagnostics because is a single, useful metric for communicating the precision and recall of our neural network. The F1-score, in simple terms, is merely the harmoic mean of the precision and recall of our model, returning a high values when both are also high. Looking back through the output of our neural network code we find the following:

Under the logistic regression:

```
f1-score
0: 0.80
1: 0.90
accuracy: 0.87
macro avg: 0.85
weighted avg: 0.86
```

Under the neural network:

```
  f1-score
0: 0.70
1: 0.83
accuracy: 0.79
macro avg: 0.77
```